A Tale of Many Streams: Characterizing a Hybrid Batch-Stream Production Workload in Digazu, a Data Lake Supported by Apache Kafka and Flink

Donatien Schmitz
UCLouvain
Louvain-la-Neuve, Belgium
donatien.schmitz@uclouvain.be

Luc Berrewaerts
Eura Nova
Mont-Saint-Guibert, Belgium
luc.berrewaerts@euranova.eu

Guillaume Rosinosky
IMT Atlantique
INRIA, LS2N, UMR CNRS 6004
F-44307 Nantes, France
guillaume.rosinosky@inria.fr

Sabri Skhiri
Eura Nova
Mont-Saint-Guibert, Belgium
sabri.skhiri@euranova.eu

Etienne Rivière
UCLouvain
Louvain-la-Neuve, Belgium
etienne.riviere@uclouvain.be

Abstract

Many industries rely on analyzing large volumes of combined historical and live data. A data lake facilitates these operations by supporting an integrated data ingestion, storage, replay, and analysis workflow.

A modern data lake is distributed and combines a processing engine, able to seamlessly process large volumes of *existing* data as well as continuous flows of *new* data, such as Apache Flink, with a storage infrastructure able to ingest and replay this data, such as Apache Kafka.

This use of Flink in this setting departs from the commonly agreed model of stream processing queries operating over windows of events, maintaining a bounded and relatively small state per operator. Instead, hybrid batch-stream queries typically process an existing data set in its entirety before updating results with incoming stream data, leading to a large accumulated state. Given the industry's importance of such usages, understanding their characteristics and how they differ from common assumptions in designing and evaluating stream processing systems is of utmost importance.

We present in this paper the analysis of a large-scale hybrid batch-stream workload collected from a production deployment of Digazu, a modern data lake building upon Kafka and Flink. We characterize 142 different sources of data and 129 hybrid batch-stream queries. Our analysis offers valuable insights into the nature of data and queries in typical data lake deployment, which will assist designers of such systems and associated benchmarks.

CCS Concepts

• **Information systems** → **Data warehouses**; **Data streams**; • **Computer systems organization** → **Data flow architectures**.

Keywords

data lake, stream processing, batch processing, workload characterization, Flink, Kafka

ACM Reference Format:
Donatien Schmitz, Luc Berrewaerts, Guillaume Rosinosky, Sabri Skhiri, and Etienne Rivière. 2025. A Tale of Many Streams: Characterizing a Hybrid Batch-Stream Production Workload in Digazu, a Data Lake Supported by Apache Kafka and Flink. In *The 19th ACM International Conference on Distributed and Event-based Systems (DEBS '25), June 10–13, 2025, Gothenburg, Sweden.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3701717.3734462

DEBS '25, Gothenburg, Sweden

ACM ISBN 979-8-4007-1332-3/25/06
https://doi.org/10.1145/3701717.3734462

1 Introduction

A data lake integrates data ingestion, storage, and analysis and enables informed business decisions [38]. As a logically centralized platform, it allows business analysts to access in a single location historical and newly-arriving data, or *sources*, and execute processing queries to extract business

intelligence (following the "Extract-Load-Transform" pattern [25]). The results of queries are then stored in the data lake, forming new sources. A data lake facilitates the complete lifecycle of data management, from metadata handling and data governance to the orchestration of queries [48].

Logically centralizing data and processing emerged via the data *warehouse* concept for structured, relational data [23]. Data lakes extend this idea to unstructured data and large-scale settings. They rely on *scale-out* backends for storage and processing.

Early data lakes focused on at-rest data, prominently using Apache HDFS [33] for storage and Hadoop [59] for processing. Apache Spark [60] added support for iterative and interactive queries. Latest-generation data lakes integrate support for stream data. The ability to quickly react to incoming data and update the output of *persistent* queries is a key business asset [11].

The couple formed by Apache Kafka [47] and Apache Flink [21] is probably the most popular backend today for high-performance data lakes integrating fault-tolerant, high-performance storage and querying [2, 15, 19]. Notably, Flink advertises the support for hybrid batch-stream processing queries, i.e., "*Stream and batch processing in a single engine*" [12]. Such hybrid queries enable the processing of large volumes of historical data replayed from cold storage before dynamically processing live data and updating the results.

Traditional stream processing benchmarks [9, 27, 31, 45, 55], in line with assumptions made in the design of distributed stream processing engines [30, 35, 39, 58], focus on stream-only queries that operate over windows of events [57] and where the size of its window bounds the state per query operator. In contrast, hybrid batch-stream processing queries, often compiled from traditional SQL statements, tend to accumulate a large state, only bounded by the combined volume of at-rest and live data. Batch-only, relational benchmarks such as TPC-H [63] and TPC-DS [37] have the opposite limitation. They only consider at-rest data and give no information about the long-lived, incremental processing of incoming stream data.

Despite its practical importance in the industry, hybrid batch-stream processing remains poorly documented and understood.

Objectives. Our focus in this paper is on analyzing the characteristics of an actual data lake production workload. For this, we study an industrial deployment of Digazu [40], a modern data lake solution commercialized by Eura Nova. Digazu streamlines data management and interrogation workflows. It provides simple graphical interfaces for data management and executing hybrid batch-stream queries, expressed as SQL queries on streams of historical and new data. For its back-end, Digazu leverages scale-out deployments of Kafka and Flink. This backend is deployed on a private or public cloud using the Kubernetes container management system [3].

The deployment we observe is at a large international company.[1] This company deploys complex business analytics on multiple IoT data streams and real-time data flows resulting from customers' and employees' interactions with their IT infrastructure. Many company users (e.g., data scientists, and business experts) access this Digazu deployment to run various queries. These queries target either historical data recorded in Kafka, stream data ingested live by Kafka soon after its production, and in the vast majority of the cases a combination of both.

We instrumented the back-end of this Digazu instance (i.e., Kafka and Flink on Kubernetes). We collected application-level metrics about data sources and queries for a representative period of 24 hours, together with system-level metrics about the configuration of these systems (e.g., scale-out configurations used by Flink queries).

Our objective in analyzing the 129 queries and 142 data sources reported in our traces is to understand better the nature of a hybrid batch-stream workload in a modern data lake. We wish to inform researchers and industry practitioners of the volume, diversity, and distribution of storage and computational loads such a workload represents. While similar characterizations studies exist for data warehouses [22, 49] and relational queries [29, 43], there is, to the best of our knowledge, no published analysis of a modern production data lake supporting hybrid batch-stream queries. Stream processing benchmarks [9, 27, 31, 45, 55] are based on synthetic applications that serve the purpose of evaluating system designs [13, 28, 50]. They do not integrate data-at-rest processing in these queries and are associated with small, bounded state. Furthermore, they do not capture information such as the distribution of load *across* queries or the general diversity of queries and data sources.

While the focus of this paper is on characterizing a data lake workload, we believe our work can motivate research on more performant and/or more cost- and energy-effective batch-stream processing backends, e.g., via multi-query scheduling [6, 41, 62] or new execution models such as serverless processing [8, 53].

Outline. We present the necessary notions about Kafka and Flink in Section 2. We discuss our collection methodology and the observed metrics in Section 3. Our analysis is in Section 4. We review related work in Section 6 and conclude in Section 7.

[1] While this client company authorized this observation and the publication of our results, our confidentiality agreements do not allow us to reveal its name or make available the observed data, even after pseudonymization.

2 Background

Digazu [40] is a data lake platform. It integrates user interfaces and data management/governance solutions and streamlines the deployment and configuration of complex data storage and processing workflows. As this paper focuses on the workload experienced by the Digazu back-end, we detail in this section the notions specific to Kafka and Flink useful for the remainder of the paper.

Apache Kafka. Kafka [47] is a high-performance stream platform designed for fault tolerance and horizontal scalability. A Kafka deployment uses several storage servers, or *brokers* (each hosting multiple hard drives or SSDs). Datasets in Kafka are streams of *events* grouped as *topics*. Each topic is partitioned, typically using a hash function over event identifiers. Each partition is stored by several *brokers* for fault tolerance. A Kafka client connects to one or more partitions to receive new events or replay events starting from a specific index. Events in Kafka are structured according to a *schema*, i.e., a pre-determined number of fields with a given type. Kafka supports basic processing, e.g., using KafkaStreams [7] but Digazu, like other systems [56], uses Flink as an external platform for processing data.

Apache Flink. Flink [12] is a distributed stream processing engine designed for fault tolerance and horizontal scalability.

A Flink query is expressed as a graph of *logical operators* (**LO**). The graph starts with one or more *source* LO. Events received from sources flow through the graph of LOs and produce other events. A *sink* LO collects results and saves them to a destination, e.g., a Kafka topic. While some LO are stateless (e.g., transformations such as a `map` or `filter`), others maintain state (e.g., `join` or `group-by/aggregate`) [10]. The size of the state is an important factor in the performance of a Flink query and depends on the type of operator, the nature of the data, the locality of operations, etc. Following the traditional stream query model, operators can compute over a window of events [57]. In this case, an operator's state depends only on events previously seen in the current window. Computation can also be unbounded, i.e., an operator's state may depend on all the previously seen data. Unbounded (i.e., non-windowed) operators are standard in batch-processing queries that aggregate or filter an entire data set. Flink is advertised as supporting both batch and stream workloads [12].

Flink allows programmers to define LO graphs by code. Another popular approach, and the one used by Digazu, is to use the Table API [20] that allows submitting queries expressed in a superset of SQL. Flink's Table API compiles the SQL query to several LOs corresponding to classical relational logic operators.

Executing a query in Flink deploys several *physical tasks* over a fleet of *task managers* (i.e., worker nodes). An operator may map to several tasks to support parallel execution [13, 50]. When an operator maps to two or more tasks, its input data stream is *partitioned*, using as partitioning key a specific field in the events' schema (e.g., for a `group-by` operator, the field used for grouping) [10].

Upon submission to a Flink cluster, the graph of LOs is not deployed as is but instead optimized to a graph of *physical operators* (**PO**). LOs that are consecutive in the graph can be merged into a single PO when they use streams partitioned by the same field. The merging of operators is critical for Flink query performance, as unmerged LOs lead to costly serialization/deserialization between their supporting POs. Another critical factor impacting performance is the presence of *skew*, i.e., unbalance in the popularity of partitioning keys [34, 39, 50]. If the distribution of these keys is unbalanced, then one or a subset of the physical tasks supporting the PO may be a bottleneck, regardless of its degree of parallelism.

3 Methodology

We detail how we collected our workload trace, including constraints and limitations due to the industrial production setting.

Collection of traces. We instrumented the production deployment of Digazu to collect information from Kafka and Flink operation and at the system/infrastructure level.

The complete infrastructure runs as a collection of containers managed with Kubernetes [3]. For this particular client, Digazu is deployed on a cluster of 10 nodes, each with 32 CPU cores and 64 GiB of RAM. The cluster hosts 100 *task managers*, each equipped with 2 CPU cores and 2 GiB of RAM. Each task manager can host up to 4 LOs. Three Kafka brokers are deployed, each with 4C PUs, 4 GiB of RAM, and a 3 TiB storage. For both system- and application-level metrics, we leveraged Prometheus [4] for collection and storage. Our trace spans one full day (24 hours) of data (from 10 am to 9:59 am the next day). This observation period was chosen to represent a typical business day for the company. The resulting trace contains both static information (e.g., the structure of submitted queries) and dynamic information collected as time-series aggregates by Prometheus (e.g., the rate, in events per second, processed by a Flink task). All time series have a periodicity of 10 seconds.

Collected metrics and anonymization. We collect metrics from two main origins: (1) data sources from Kafka, and (2) queries running on Flink. This data collection follows strict data access and anonymization rules agreed with the company. This impacts the completeness of the data compared to clear-text data collection, but it was a necessary compromise for the realization of our study.

Topics. The data lake host 466 topics, some of which are used as data sources and others as sinks. The 129 queries recorded in the trace access 142 of these different topics as sources. These topics correspond to a large diversity of sources within the company, e.g., IoT streams, live events from user interactions, or the integration of existing data silos (e.g., relational databases). They contain a mix of at-rest and live data. Although we do not have direct access to the nature of the sources, i.e., statistics on the events, we were able to retrieve their total volume of at-rest data. We can observe when a hybrid or batch query replays complete topics (i.e., from index 0) and infer how many events were processed as part of such an initial batch phase. Following this phase, we can observe the topic's streaming rate of newly-arriving events. We do not have access to the cleartext schemas used for events stored in Kafka topics, but we can infer most of this information from queries. Topic names are made visible by the TableAPI, along with their field names. We record the *number* of fields (schema size) using source LOs' readings. The *type* of each field is unknown to the source logical operator (LO), e.g., `int`, `string`, `date`, `boolean`, etc. However, we can infer this field type in 80% of the cases by analyzing the query graph and its LOs. Indeed, many LOs explicitly enforce a field type for their operation from which we can make this deduction.

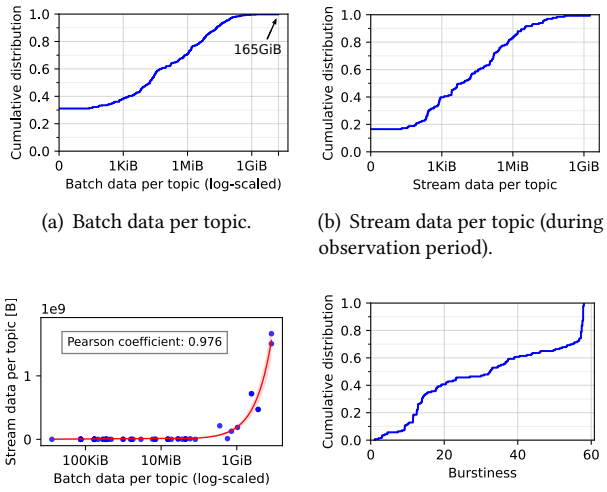
Queries. The 129 queries we observe from 92 users are all expressed in SQL using Flink's Table API. For each query, we extract its logical structure, i.e., the set of LOs, their nature (e.g., source, sink, `filter`, `join`, ...), their connections in the query graph, and which ones need to maintain state. In addition, we record the physical query implementation, i.e., after Flink compiles it for execution: we record the number of physical operators (POs), which LOs they aggregate, and which key (field identifier) is used for keyed streams crossing PO boundaries.

All queries ran over the full, 24-hour observation period. We do not have access to the initial submission time or responsible user.

The query deployment maps each PO to several physical tasks. This degree of parallelism is chosen by platform administrators (i.e., there was no elastic scaling [13, 50] during the day of observation). We record this degree of parallelism for each PO.

For each physical task of a running query, we collect several time series about its execution. The output rate and written Bytes give the average number of events and volume of data this task outputs per second. Aggregating over its tasks yields the same metric for a complete PO. Input rate and read Bytes function similarly.

Some POs contain one or more stateful LO(s), making them *de facto* stateful. Knowing the state size of a stateful PO's task(s) is interesting for memory provisioning and capacity planning [5, 52]. Collecting the exact state size requires instrumenting the state backend of Flink (RocksDB [18]), which we evaluated as having a too large impact to be deployed on this production system. We build upon the fact that Flink queries' states are periodically checkpointed to disk [10] for recovery after faults and reconfigurations. The size of incremental checkpoints is an indicator of the state size and its evolution over time. We collect it together with the checkpoint collection time.



(a) Batch data per topic.

(b) Stream data per topic (during observation period).

(c) Correlation between batch and stream data per topic. (P_c = 0.976)

(d) Distribution of stream data burstiness.

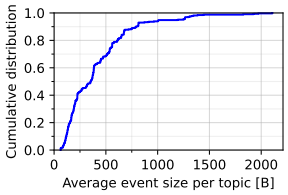
Figure 1: Data volumes across Kafka topics.

4 Workload Analysis

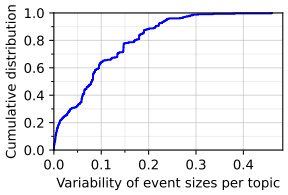
We analyze our workload trace in three phases. First, we explore the nature of *data* stored in Kafka (§4.1). Then, we characterize the structure of *queries* (§4.2) and their execution (§4.3). When discussing our results, we extract key takeaway information and compare them with the assumptions made in the design of Nexmark [55], an academia-contributed benchmark extensively used for performance validation of stream processing solutions [5, 30, 52]. We further compare our observations with assumptions and characteristics of other benchmarks in Section 5.

4.1 Data in and from Kafka

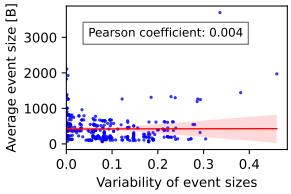
The Digazu data lake hosts 466 topics of varying sizes, with the largest topic containing 165 GiB of data. Among those topics, 285 (~ 61%) are direct data sources, including the results of other queries, while the remaining 181 (~ 39%) are not used as sources for other queries. A total of 142 Kafka



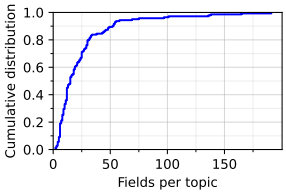
(a) Distribution of average event sizes.



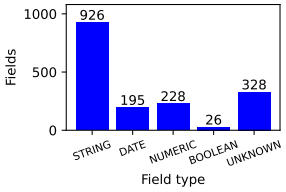
(b) Dist. of event sizes variability.



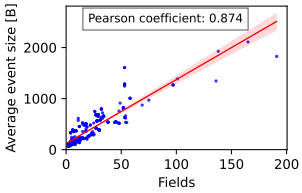
(c) Correlation between average event size and its variability. (P_c = 0.004)



(d) Distribution of schema size (#fields).



(e) Frequency of field types.



(f) Correlation between event schema size and average event size. (P_c = 0.874)

Figure 2: Events characteristics across Kafka topics.

topics were used by *at least* one query during our 24-hour observation period. During this period, these topics received a total of 19,607,011 events for 20 GiB of data. Around ~ 10% of the topics (14 topics) are the results of other queries.

Figure 1 presents the volume of batch and stream data per topic. Figure 1(a) shows the volume of batch data per topic, which is the data that was already present in Kafka before our observation period. We see that the majority of topics have a small amount of batch data, with only a few topics accounting for the majority of the data. Figure 1(b) shows that ~18% of the topics did not ingest new events during the observation period, while a single topic, containing the largest volume of batch data, has ingested ~16 GiB worth of events. Overall, we identify a significant disparity in the volume of stream data per topic with a median value of ~66 KiB.

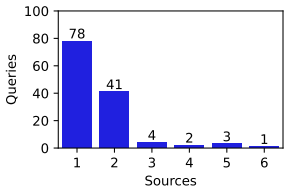
We further evaluate a possible linear correlation between the volume of stream data generated by topics and their initial batch data size. The linear correlation between the two dimensions is measured using a Pearson coefficient, which we note P_c in the rest of this paper. Ranging from -1 to 1, P_c maps from a perfect negative linear relation between two continuous variables to a perfect positive one. $P_c = 0$ indicates no linear relation. Figure 1(c) shows the scatter plot of the two metrics (the X-axis is displayed in log scale for visibility) for the 142 topics used during the observation period. A P_c of 0.976 indicates a strong linear correlation between the two metrics.

We then analyze the topics' "*burstiness*", i.e., their tendency to produce events in bursts rather than continuously. Figure 1(d) shows the CDF of a burstiness metric that we define as the peak-to-average ratio, i.e., the ratio between the maximum throughput observed in a 5-minute time window and the average throughput. A burstiness value of 1 indicates a perfectly steady flow of events. Bursty patterns have greater burstiness values. The results show that the large majority of topics produce events in bursts. We evaluate a possible linear correlation between the volume of data generated by topics and their burstiness. A $P_c = -0.13$ indicates the absence of a clear linear correlation.

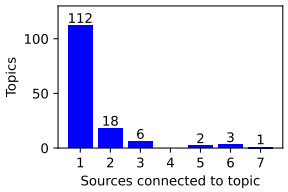
In Figure 2, we further analyze the nature of events in these topics. Figure 2(a) shows the average event size distribution across topics, ranging from 60 B to 3,700 B, with a median of 353 B. To understand whether this small average event size does not "hide" an unbalanced size distribution mixing small and large events, Figure 2(b) plots *variability*, i.e., the ratio between standard deviation and average of event sizes. For 60% of the topics, event size has low variability (< 10%). For the remaining 40%, this variability remains very moderate (< 50%). For comparison, Nexmark [55] uses events from three topics of average sizes 100 B, 300 B, or 500 B, with 20% variability.

We evaluate the correlation between the average event size and variability (normalized as a fraction of this average size). Figure 2(c) is a scatter plot showing, for each topic, a point linking its average event size and its variability. In addition to P_c, we plot the linear regression using a red line along with a 95% confidence interval using a red-shaded zone. The coefficient between average event size and topic size variability is $P_c = 0.004$, indicating no linear relation between the two metrics.

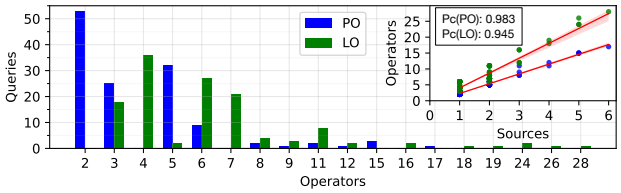
We now analyze the schema characteristics across the 142 topics. Figure 2(d) presents the distribution of the number of fields. Schemas range from 2 to 191 fields, with a median of 15. In contrast, Nexmark's three topics use schemas of 7, 8, and 10 fields. The distribution of the 80% of types information we could infer from queries (as detailed in Section 3) is given by Figure 2(e). Most identified field types are strings, followed by numerical values, dates, and booleans. The correlation between the average event size and the schema size is given by Figure 2(f), with a strong positive linear correlation (P_c =



(a) Sources per query.

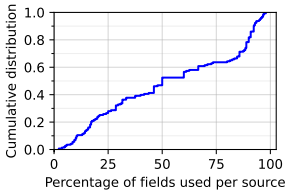


(b) Sources per topic.

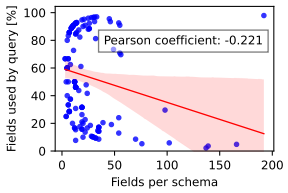


(c) POs and LOs per query. The scatter plot on the right shows a linear correlation between the number of POs/LOs and sources per query.

Figure 3: Queries sources and graph size.



(a) Distribution of useful fields per query. The X-axis represents the pourcentage of fields present in the source effectively used by the query.



(b) Correlation between percentage of *useful* fields and initial schema size. ($P_c = -0.221$)

Figure 4: Statistic on topic schema usage.

0.874). The primary factor contributing to event size appears to be the number of fields rather than some particularly large values over a subset of these fields. Nexmark fixes the size of events to 500 B by padding data in a field present in all schemas and named "extra". This does not correspond to the variety of sizes we observe in production and loses any aspect of correlation between the schema size and the event size, the latter being constant.

Takeaway 1. Data received in the data lake is highly heterogeneous in volume but consists of relatively small events with limited size variability. The principal factor for event size is the size of the schema, possibly featuring many fields. Event size and variability are similar to the values used by Nexmark. Still, our workload heterogeneity in schema size and volume per topic is significantly higher, as is the actual number of actual data sources (142 *vs.* 3).

4.2 Queries' static properties and structures

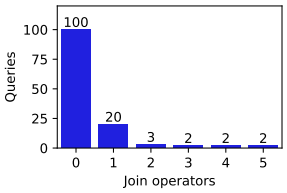
We now analyze the nature of the 129 queries submitted to Flink. In this subsection, we first look at the structure of queries, both upon submission (i.e., the graph of LOs) and after compilation and optimization by Flink (i.e., the resulting set of POs).

We first look at the source Kafka topics used by queries (hereafter denoted as "sources" for brevity). Figure 3(a) shows that 78/129 (~60%) of the queries use a single source. 41 (~32%) use two sources, and only 10 (~8%) use more, up to 6 sources. Figure 3(b) shows the number of queries reading from each of the 142 Kafka topics (note that, as we were not allowed to list all topics but only infer those used by queries, each topic is sourced by at least one query in the trace). 112/142 (~79%) of the topics are used by a *single* query. The other 30 topics are used by a small number (up to 7 for a single topic) of queries. This may be a surprising result: a classical expectation (the 80/20 rule) is that a large number of queries use a small number of topics. In fact, the largest topic by volume is accessed by a single query, and there is no clear correlation between the volume of data and the number of queries using a topic.

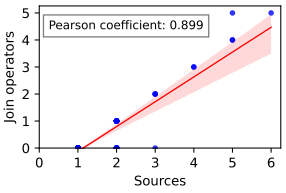
Figure 3(c) presents the distribution of the queries graph size before and after operator merge (i.e., number of LOs and POs). We observe that merging effectively reduces the number of physical operators that have to be deployed using physical tasks in the Flink cluster: the majority of queries have less than 4 POs, and the distribution is shifted leftwards. Queries have from 3 to 28 LOs but at most 17 POs.

The main factor preventing the merge of LOs into POs is the presence of stateful operators in the query, particularly `join` LOs, as the two flows consumed by such operators are generally not from the same original source and/or not partitioned using the same key. Multi-source queries are typical of this case. The embedded scatter plot in Figure 3(c) relates the query graph sizes to the number of sources, establishing a strong linear correlation in both cases ($P_c = 0.945$ for LOs and $P_c = 0.983$ for POs). Most queries with 2 or 3 POs (53/79, or ~68%) are *stateless* ones with a single source, i.e., performing simple transformations on individual events. In fact, we identify 8 queries (~6%) that only filter events and 45 queries (~35%) either transforming or projecting events.

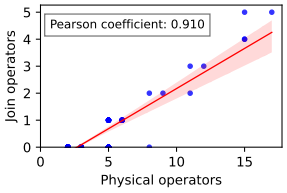
We then analyze what we call the percentage of *useful* fields of a source. While reading data from a Kafka topic, Flink fetches the whole schema of the records even if only a subset of the schema is necessary for the query. This results in unnecessary network traffic and larger deserializations. We define the ratio of *useful* fields as the ratio between the number of fields necessary to execute the query's logic and the number of fields in the initial schema. Figure 4(a) shows



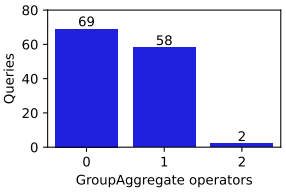
(a) `join` LOs per query.



(b) Correlation between number of `join` LOs and sources. ($P_c = 0.899$)

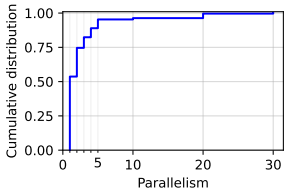


(c) Correlation between number of `join` LOs and number of POs. ($P_c = 0.910$)

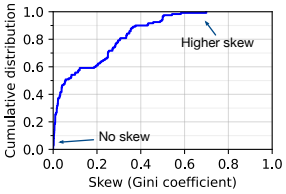


(d) `group-by/aggreg.` LOs per query.

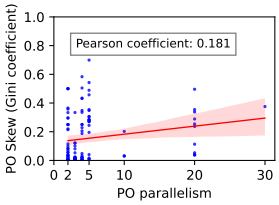
Figure 5: Statistics on queries' operators.



(a) CDF of the degree of parallelism across all POs of all queries.



(b) CDF of the skew metric of all POs with parallelism > 1.



(c) Correlation between parallelism degree and skew for all POs with parallelism > 1. ($P_c = 0.181$)

Figure 6: Runtime statistics of queries.

the distribution of ratios for each source. Note that some sources did not expose the schemas to Flink and were omitted from the figure. The results indicate that the majority of sources transfer unnecessary data with a median of 50% of fields being *useful*. We also note that none of the queries used all the fields of a source. Finally, we analyze a possible correlation between the percentage of *useful* fields and the initial schema size. Figure 4(b) shows a scatter plot of the two metrics. The correlation coefficient is $P_c = -0.221$, indicating a weak negative linear correlation.

We further analyze the operators used in the queries, focusing on *stateful* operators. None of the stateful operators uses windowed operations, i.e., all compute over the entire data set from their corresponding sources, matching the hybrid batch-stream query model discussed in our introduction.

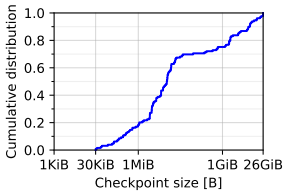
Our results are in Figure 5. 76/129 (~59%) of the queries have at least one stateful LO (and, therefore, at least one stateful PO). 29/129 (~22%) of the queries have at least one `join` LO (Figure 5(a)); only a few queries have 2 or more, and none has more than 5. Unsurprisingly, there is a strong linear correlation ($P_c = 0.910$) between this number and the number of sources as `join` LOs mostly consume data from distinct streams (Figure 5(b)). Note that, however, some queries have more `join` LOs than sources, as a single topic may be *split* by different `filter` LOs before joining the two resulting substreams. Reading events of different natures from a unique source before splitting streams as part of the Flink query is, in fact, the approach used by common implementations of Nexmark. The strong linear correlation between the number of `join` LOs and the total number of POs in the optimized query graph (Figure 5(c)) concurs with our previous observation that such operators are the principal obstacle to operator merging and, as a result, a key factor in query complexity and deployment cost. Finally, we observe the distribution of the number of `group-by/aggregate` LOs (i.e., collecting data for individual keys in a partitioned stream and computing `sum, count, min, max, ...` aggregates for each key). 69/129 queries (~53%) have no such LO, while the rest (60/129) have one or, for only two queries, two of them.

Takeaway 2. Queries tend to be relatively complex and include stateful operators. Operator merging is successful in many cases but limited by `join` operators, making queries with multiple sources intrinsically more complex to deploy and run. On the other hand, the distribution of sources' popularity is relatively balanced.

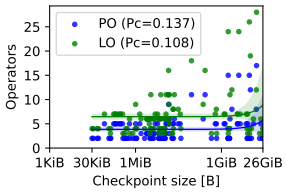
4.3 Dynamic query statistics

In this final phase, we analyze the runtime behavior of queries. We report some configuration choices made in the observed deployment, e.g., on the parallelism of POs. Then, we study the load distribution between parallel tasks of a given operator to measure event skew. Last, we study state size indirectly by analyzing checkpointing metrics.

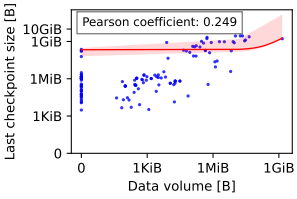
The POs of the 129 queries are deployed over 1,365 tasks. Figure 6(a) presents the distribution of tasks per PO. More than half of POs (~53%) are not scaled out, i.e., they have a parallelism of 1. Conversely, some POs run with a parallelism



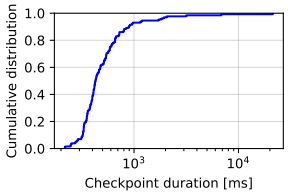
(a) CDF of last observed cumulative checkpoint size per query.



(b) Correlation between checkpoint size and number of POs/LOs.



(c) Correlation between last checkpoint size and data volume ingested during observation period (log scales). (P_c = 0.249)



(d) CDF of the average checkpoint duration per query.

Figure 7: Checkpointing and state statistics.

of 10, 20, or even 30 tasks. The engineers provisioned tasks statically to cope with the bursts of data associated with the (re)start of queries and the system does not use elastic scaling. As a result, operators' tasks may be underutilized outside of these bursts. Unfortunately, we cannot access systems-level metrics to validate this intuition.

We continue by analyzing data skew, i.e., the unbalance in input data distribution across a PO's tasks. We measure this skew using the Gini coefficient [36], a metric commonly used in economics and social sciences and also to measure the fairness of a load distribution [24, 44]. The Gini coefficient, which we note G_c hereafter, is computed on the distribution of rates (in events) across all tasks of a PO. G_c ranges from 0 for a perfectly balanced distribution to 1 for a distribution in which one task gets all the input traffic. Figure 6(b) shows the skew metric G_c distribution for all POs with more than one task. A minority of POs have a highly skewed distribution of inputs over their tasks, yet it remains for most PO higher than what we calculated for the skew generated by Nexmark's event source, with a skew of G_c =~0.018 over its dominant `Bid` events. Figure 6(c) shows the correlation between the parallelism of POs with more than one task, and their skew metric G_c. With P_c = 0.181, there is a modest linear correlation between the two variables: Larger POs are also subject to skew, and slightly more so than smaller ones.

We now analyze statistics about checkpoints in Figure 7. Checkpoints are taken at regular intervals, i.e., every 10 minutes in our trace. Snapshotting the state of a query requires waiting at each of its PO for specific *barrier* events from all upstream flows and storing a local snapshot before resuming processing [10]. We collect two metrics: the combined size of these local PO state snapshots and the downtime incurred for the query by collecting them.

Figure 7(a) plots the CDF of the last observed checkpoint size across queries, ranging from ~30 KiB to ~26 GiB, and with a median value of ~10 MiB. The cumulated size of all snapshots is ~276 GiB. Even queries with only stateless LOs have a small snapshot of a few KiB, as their source PO must record the offset in their respective Kafka topic partition(s). We analyze the correlation between checkpoint size and the number of LOs and POs in queries. Figure 7(b) is a scatter plot for both POs (blue) and LOs (green). There is only a very small linear correlation between a query's graph complexity and its state size (P_c=0.108 for LOs and P_c=0.137 for POs). Figure 7(c) studies the correlation between the data volume processed by the query during our observation period and the size of the last checkpoint (note the log scale on both axes). Importantly, we point out that queries are long-lived and may have been deployed long before our observation period and accumulated state since then. We observe a slightly higher linear correlation (P_c=0.249) than for query complexity. Some low-volume (over our 24-hour observation period) queries have relatively large snapshots; these queries accumulate all events from source(s) in some `join` LOs.

Figure 7(d) presents the CDF of the average checkpointing duration across all queries. When a snapshot is taken, processing halts, resulting in downtime, possibly impacting queries with latency constraints (in this particular Digazu deployment, the expectation is that events be processed within one *minute* of their ingestion by the system, and there is no strict latency constraints that would require other forms of fault-tolerance [51]). The large majority (~ 92%) of the queries have an average checkpoint duration of less than 1 second. However, some queries with larger states require multiple seconds and as much as ~24 seconds average in the worst case. There is a moderate linear correlation (P_c = 0.332) between the checkpoint size and duration, but not a perfect one. Indeed, the checkpoint duration is primarily affected by the volume of new values in RocksDB, but these values may either replace (same key) or complement (new keys) those already stored, influencing how the checkpoint size evolves.

Takeaway 3. Runtime observations show that there is a significant amount of skew in data received by the queries' POs. This confirms the observation that skew is a major challenge for performance in stream processing systems [34, 39]. Nexmark's default configuration introduces some modest skew, but below our observations. The checkpoint size (accumulated state) of queries may be quite significant even

Characteristics	**Nexmark [55]**	**ESPBench [27]**	**YSB [1]**	**TPC-H [63]**	**TPC-DS [37]**	**Our workload**
Execution mode	S	S	S	B	B	H
Queries	22	5	1	22	103	129
Sources	3	7	1	8	24	142
Schema sizes (min;median;max)	7;8;10	3;6;19	7;7;7	3;7.5;16	3;14;34	2;15;191

Table 1: Comparison to standard benchmarks. (S: Stream, B: Batch, H: Hybrid)

when the volume of data in the observation period is low, highlighting that very-long-lived queries with possibly large states but low instantaneous volumes are more common than short-lived, high-throughput queries.

5 Discussion

Many synthetic benchmarks exist for measuring stream processing performance [1, 9, 27, 31, 37, 45, 55, 63]. We compare in Table 1 our workload to five common benchmarks. Stream benchmarks Nexmark [55], ESPBench [27], and the Yahoo! streaming benchmark, YSB [1] solely consider stream queries, i.e., where data is processed at a rate defined by the benchmark or configured by the user. These benchmarks focus on window-based processing of stream data and overlook the impact of an ever-increasing, unbounded state. Batch benchmarks TPC-H [63] and TPC-DS [37] process events from disk as fast as the processing engine allows.

Our hybrid batch-stream workload contains more queries and sources than benchmarks of the Stream type, and its schema sizes are larger. However, the volume of data processed per query and day is lower. These benchmarks also include much less imbalance in data sources, be it in terms of skew *within* each source (popularity of keys over their events) or volume distributions *across* sources. TPC-DS [37] is the closest in terms of query characteristics, but being a Batch workload, it does not define relative rates for its different sources and is not suitable for hybrid batch-stream systems.

Much of the research on stream processing focuses on efficiently supporting a single, high-rate query [13, 28, 50]. Our workload contains multiple small-volume queries that still consume state and occupy Flink tasks and resources. Resource sharing as proposed in Astream [32], state offloading using RDMA [16], or scale-up processing engines [61] are certainly options to reduce these costs. Multi-query scheduling [6, 41, 62] should consider the imbalance between queries data volume and the "long tail" of low-traffic queries. A trend is to decouple state storage and event processing with a serverless stream processing model [8, 42, 53]. We believe a hybrid model combining such serverless operators with dedicated resources for high-volume queries is a direction worth exploring.

6 Related work

We discussed stream processing benchmarks and execution models, mostly contributed by academia, in the previous section. In this section, we discuss industry-focused papers on data processing and storage systems characterization.

IBM's Garcés-Erice *et al.* [22] report statistics about a data lake built over Hadoop and running SQL queries over large, static data sets. The authors point out the difficulty of planning the resource consumption of such queries, a problem known as capacity planning and that StreamBed [52] targets for Flink. Two other notable works from IBM are Yu *et al.* [43]'s seminal paper on relational database workload characterization and the REDWAR workload analysis tool for DB2, and Hsui *et al.* [29]'s comparison of a production database workload and the TCP benchmarks.

TaoBao, an e-commerce company, reports a workload analysis of a 2000-node cluster supporting Hadoop [49]. Researchers from Intel characterize the memory performance of large scale data processing workloads [17]. Chen *et al.* [14] study the characteristics of MapReduce workloads across different e-commerce actors such as Facebook and Cloudera.

Halevy *et al.* [26] detail Goods, Google's data lake, but do not detail its workload. Microsoft researchers report on the Azure Data Lake Store [46], a scale-out file system for Hadoop and MapReduce workloads. Performance results are based on synthetic benchmarks; the authors do not analyze production workloads.

Talluri *et al.* [54] analyze the workload of a large-scale data processing cluster over a period of 6 months. Their work provides valuable information about the volumes of data processed and their distribution over time but lacks details about the nature of the queries.

7 Conclusion

We analyzed a production hybrid batch-stream workload of Digazu, a data lake building upon the popular Apache Kafka and Flink systems. This analysis is motivated by the lack of documentation of similar real-world workloads in the literature. Our findings show that this workload is characterized by many streams and queries that are highly heterogeneous in size and complexity. We identified that commonly used

synthetic benchmarks do not always represent our production workload. They consider only a few queries and streams or do not include sufficiently unbalanced data sources (skew). The existence of many low-volume, high-state persistent queries makes a case for re-designing stream processing engines and decoupling state and computation for such queries. We hope our study will benefit designers of workloads and benchmarks and shed light on stream processing and storage engine system requirements in an actual industrial use case.

Acknowledgments

This research was funded by the Walloon region (Belgium) through the Win2Wal project "GEPICIAD" and by a gift from Eura Nova.

References

[1] 2015. Yahoo Streaming Benchmark. https://developer.yahoo.com/blogs/135370591481/?guccounter=1.

[2] 2024. Aiven. https://aiven.io

[3] 2024. Kubernetes. https://kubernetes.io.

[4] 2024. Prometheus. https://prometheus.io.

[5] Pratyush Agnihotri, Boris Koldehofe, Paul Stiegele, Roman Heinrich, Carsten Binnig, and Manisha Luthra. 2024. ZeroTune: Learned Zero-Shot Cost Models for Parallelism Tuning in Stream Processing. In *40th IEEE International Conference on Data Engineering (ICDE)*.

[6] Mutaz Barika, Saurabh Garg, Andrew Chan, and Rodrigo N Calheiros. 2019. Scheduling algorithms for efficient execution of stream workflow applications in multicloud environments. *IEEE Transactions on Services Computing* 15, 2 (2019), 860–875.

[7] Bill Bejeck. 2024. *Kafka Streams in Action*. Simon and Schuster.

[8] Philip A Bernstein, Todd Porter, Rahul Potharaju, Alejandro Z Tomsic, Shivaram Venkataraman, and Wentao Wu. 2019. Serverless Event-Stream Processing over Virtual Actors.. In *Conference on Innovative Data Systems Research (CIDR)*.

[9] Maycon Viana Bordin, Dalvan Griebler, Gabriele Mencagli, Cláudio FR Geyer, and Luiz Gustavo L Fernandes. 2020. DSPBench: A suite of benchmark applications for distributed data stream processing systems. *IEEE Access* 8 (2020), 222900–222917.

[10] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State management in Apache Flink®: consistent stateful distributed stream processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.

[11] Paris Carbone, Marios Fragkoulis, Vasiliki Kalavri, and Asterios Katsifodimos. 2020. Beyond analytics: The evolution of stream processing systems. In *ACM SIGMOD international conference on Management of data (SIGMOD'20)*. 2651–2658.

[12] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* 38, 4 (2015).

[13] Valeria Cardellini, Francesco Lo Presti, Matteo Nardelli, and Gabriele Russo Russo. 2022. Runtime Adaptation of Data Stream Processing Systems: The State of the Art. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–36.

[14] Yanpei Chen, Sara Alspaugh, and Randy Katz. 2012. Interactive Analytical Processing in Big Data Systems: A Cross-Industry Study of MapReduce Workloads. *Proceedings of the VLDB Endowment* 5, 12 (2012).

[15] Hu Zheng (Alibaba Cloud). 2021. Building an Enterprise-Level Real-Time Data Lake Based on Flink and Iceberg. https://www.alibabacloud.com/blog/building-an-enterprise-level-real-time-data-lake-based-on-flink-and-iceberg_597755

[16] Bonaventura Del Monte, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2022. Rethinking stateful stream processing with RDMA. In *ACM SIGMOD international conference on Management of data (SIGMOD'22)*. 1078–1092.

[17] Martin Dimitrov, Karthik Kumar, Patrick Lu, Vish Viswanathan, and Thomas Willhalm. 2013. Memory system characterization of big data workloads. In *2013 IEEE International Conference on Big Data (BigData'13)*. 15–22.

[18] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. RocksDB: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Trans. on Storage* 17, 4 (2021).

[19] Allen Wang (DoorDash). 2022. Building Scalable Real Time Event Processing with Kafka and Flink. https://doordash.engineering/2022/08/02/building-scalable-real-time-event-processing-with-kafka-and-flink/

[20] Apache Flink. 2024. Table API. https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/dev/table/tableapi/

[21] Apache Software Foundation. 2025. Apache Flink. https://flink.apache.org

[22] Luis Garcés-Erice, Sean Rooney, and Zoltán A Nagy. 2020. Analysis of SQL Workloads on an Enterprise Datalake. In *2020 IEEE 13th International Conference on Cloud Computing (CLOUD'20)*. 31–33.

[23] Stephen R Gardner. 1998. Building the data warehouse. *Commun. ACM* 41, 9 (1998), 52–60.

[24] Gajendra Singh Gurjar and Sharda Chhabria. 2015. A review on concept evolution technique on data stream. In *2015 International Conference on Pervasive Computing (ICPC'15)*. IEEE, 1–3.

[25] Sarah Myriam Lydia Hahn, Ionela Chereja, and Oliviu Matei. 2022. Evaluation of transformation tools in the context of NoSQL databases. In *Intelligent Systems and Applications: Proceedings of the 2021 Intelligent Systems Conference (IntelliSys) Volume 2*. Springer, 146–165.

[26] Alon Y Halevy, Flip Korn, Natalya Fridman Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. 2016. Managing Google's data lake: an overview of the Goods system. *IEEE Data Eng. Bull.* 39, 3 (2016), 5–14.

[27] Guenter Hesse, Christoph Matthies, Michael Perscheid, Matthias Uflacker, and Hasso Plattner. 2021. ESPBench: The enterprise stream processing benchmark. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE'21)*. 201–212.

[28] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. 2014. A catalog of stream processing optimizations. *ACM Computing Surveys (CSUR)* 46, 4 (2014), 1–34.

[29] Windsor W Hsu, Alan Jay Smith, and Honesty C. Young. 2001. Characteristics of production database workloads and the TPC benchmarks. *IBM Systems Journal* 40, 3 (2001), 781–802.

[30] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. 2018. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18)*.

[31] Jeyhun Karimov, Tilmann Rabl, Asterios Katsifodimos, Roman Samarev, Henri Heiskanen, and Volker Markl. 2018. Benchmarking distributed stream data processing systems. In *2018 IEEE 34th International Conference on Data Engineering (ICDE'18)*. 1507–1518.

[32] Jeyhun Karimov, Tilmann Rabl, and Volker Markl. 2019. Astream: Ad-hoc shared stream processing. In *ACM SIGMOD international conference on Management of data (SIGMOD'19)*. 607–622.

[33] A Kala Karun and K Chitharanjan. 2013. A review on Hadoop—HDFS infrastructure extensions. In *2013 IEEE conference on information & communication technologies*. 132–137.

[34] Nikos R Katsipoulakis, Alexandros Labrinidis, and Panos K Chrysanthis. 2017. A holistic view of stream partitioning costs. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1286–1297.

[35] Jinqing Lian, Xinyi Zhang, Yingxia Shao, Zenglin Pu, Qingfeng Xiang, Yawen Li, and Bin Cui. 2023. ContTune: Continuous Tuning by Conservative Bayesian Optimization for Distributed Stream Data Processing Systems. In *VLDB*, Vol. 16. 4282–4295.

[36] Branko Milanovic. 1997. A simple way to calculate the Gini coefficient, and some implications. *Economics Letters* 56, 1 (1997), 45–49.

[37] Raghunath Othayoth Nambiar and Meikel Poess. 2006. The Making of TPC-DS.. In *VLDB*, Vol. 6. 1049–1058.

[38] Fatemeh Nargesian, Erkang Zhu, Renée J Miller, Ken Q Pu, and Patricia C Arocena. 2019. Data lake management: challenges and opportunities. *Proceedings of the VLDB Endowment* 12, 12 (2019), 1986–1989.

[39] Muhammad Anis Uddin Nasir, Gianmarco De Francisci Morales, Nicolas Kourtellis, and Marco Serafini. 2016. When two choices are not enough: Balancing at scale in distributed stream processing. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE'16)*. 589–600.

[40] Eura Nova. 2025. Digazu. https://digazu.com

[41] Dimitris Palyvos-Giannas, Gabriele Mencagli, Marina Papatriantafilou, and Vincenzo Gulisano. 2021. Lachesis: a middleware for customizing OS scheduling of stream processing queries. In *Proceedings of the 22nd International Middleware Conference (Middleware'21)*. 365–378.

[42] Matthew Perron, Raul Castro Fernandez, David DeWitt, Michael Cafarella, and Samuel Madden. 2023. Cackle: Analytical Workload Cost and Performance Stability With Elastic Pools. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–25.

[43] S Yu Philip, Hans-Ulrich Heiss, Sukho Lee, and Ming-Syan Chen. 1992. On Workload Characterization of Relational Database Environments. *IEEE Trans. Software Eng.* 18, 4 (1992), 347–355.

[44] Theoni Pitoura and Peter Triantafillou. 2007. Load distribution fairness in P2P data management systems. In *2007 IEEE 23rd International Conference on Data Engineering (ICDE'07)*. 396–405.

[45] Meikel Poess, Raghunath Othayoth Nambiar, and David Walrath. 2007. Why You Should Run TPC-DS: A Workload Analysis.. In *VLDB*, Vol. 7. 1138–1149.

[46] Raghu Ramakrishnan, Baskar Sridharan, John R Douceur, Pavan Kasturi, Balaji Krishnamachari-Sampath, Karthick Krishnamoorthy, Peng Li, Mitica Manu, Spiro Michaylov, Rogério Ramos, et al. 2017. Azure data lake store: a hyperscale distributed file service for big data analytics. In *ACM SIGMOD international conference on Management of data (SIGMOD'17)*. 51–63.

[47] Theofanis P Raptis and Andrea Passarella. 2023. A survey on networked data streaming with Apache Kafka. *IEEE access* (2023).

[48] Franck Ravat and Yan Zhao. 2019. Data lakes: Trends and perspectives. In *Database and Expert Systems Applications: 30th International Conference, DEXA 2019, Linz, Austria, August 26–29, 2019, Proceedings, Part I 30*. Springer, 304–313.

[49] Zujie Ren, Jian Wan, Weisong Shi, Xianghua Xu, and Min Zhou. 2013. Workload analysis, implications, and optimization on a production Hadoop cluster: A case study on TaoBao. *IEEE Transactions on Services Computing* 7, 2 (2013), 307–321.

[50] Henriette Röger and Ruben Mayer. 2019. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Computing Surveys (CSUR)* 52, 2 (2019), 1–37.

[51] Guillaume Rosinosky, Florian Schmidt, Oleh Bodunov, Christof Fetzer, André Martin, and Etienne Riviere. 2021. Active replication for latency-sensitive stream processing in Apache Flink. In *2021 40th International Symposium on Reliable Distributed Systems (SRDS'21)*. IEEE, 56–66.

[52] Guillaume Rosinosky, Donatien Schmitz, and Etienne Rivière. 2024. StreamBed: capacity planning for stream processing. In *18th ACM International Conference on Distributed and Event-Based Systems (DEBS'24)*.

[53] Won Wook Song, Taegeon Um, Sameh Elnikety, Myeongjae Jeon, and Byung-Gon Chun. 2023. Sponge: Fast Reactive Scaling for Stream Processing with Serverless Frameworks. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 301–314.

[54] Sacheendra Talluri, Alicja Łuszczak, Cristina L Abad, and Alexandru Iosup. 2019. Characterization of a big data storage workload in the cloud. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. 33–44.

[55] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. 2010. *NEXMark–a benchmark for queries over data streams*. Technical Report. OGI School of Science & Engineering at OHSU.

[56] Giselle Van Dongen and Dirk Van den Poel. 2020. Evaluation of stream processing frameworks. *IEEE Transactions on Parallel and Distributed Systems* 31, 8 (2020), 1845–1858.

[57] Juliane Verwiebe, Philipp M Grulich, Jonas Traub, and Volker Markl. 2023. Survey of window types for aggregation in stream processing systems. *The VLDB Journal* (2023), 1–27.

[58] Yuanli Wang, Lei Huang, Zikun Wang, Vasiliki Kalavri, and Ibrahim Matta. 2024. CAPSys: Contention-aware task placement for data stream processing *(EuroSys)*.

[59] Tom White. 2012. *Hadoop: The definitive guide.* " O'Reilly Media, Inc.".

[60] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al. 2016. Apache Spark: a unified engine for big data processing. *Commun. ACM* 59, 11 (2016), 56–65.

[61] Steffen Zeuch, Bonaventura Del Monte, Jeyhun Karimov, Clemens Lutz, Manuel Renz, Jonas Traub, Sebastian Breß, Tilmann Rabl, and Volker Markl. 2019. Analyzing efficient stream processing on modern hardware. *Proceedings of the VLDB Endowment* 12, 5 (2019), 516–530.

[62] Feng Zhang, Chenyang Zhang, Lin Yang, Shuhao Zhang, Bingsheng He, Wei Lu, and Xiaoyong Du. 2021. Fine-grained multi-query stream processing on integrated architectures. *IEEE Transactions on Parallel and Distributed Systems* 32, 9 (2021), 2303–2320.

[63] Jianyong Zhang, Anand Sivasubramaniam, Hubertus Franke, Natarajan Gautam, Yanyong Zhang, and Shailabh Nagar. 2004. Synthesizing representative i/o workloads for tpc-h. In *10th International Symposium on High Performance Computer Architecture (HPCA'04)*. IEEE, 142–142.